AI Stalemate



THE LEAD DEVELOPER FIELD GUIDE

Explain the concerns. Work out the next step with your PM.

Nic Payne

For you, the lead developer

Your PM wants to try AI. You have questions.

Your PM sees possibilities in AI. You will help turn an idea into something the team can build and support. Let's understand the business pressure, explain your concerns, and help choose a worthwhile next step together.

What are you hoping to improve?

Ask why the work matters now. A repair-request workflow keeps slipping while support staff copy everything twice. Your PM may have customer or leadership context you haven't heard. Share what you see slowing delivery too.

What is your colleague worried about?

Be specific about what worries you: review capacity, reliability, maintenance, or what this means for your work. You don't have to defend every part of today's process to care about quality. Say what you would be willing to explore.

What have you assumed about each other?

"They asked whether we could try another tool" describes what happened. "They don't care about quality" adds a motive. So does "My lead just hates change." Ask what is behind the request or hesitation.

What could you work out together?

You might agree to try something small. You might need permission, advice, or more information first. You might decide this use isn't worth pursuing. All of those can be useful answers.

The aim is to understand the opportunity and the concerns well enough to decide what to do next together.

Reading and reflection: see the companion at the end of this Guide >

Give them something they can picture.

Let's start with something your colleague can picture. If this went well, whose work or experience would get better, and how?

Describe an ordinary day

Our support team copies every repair request into two systems. Sometimes they miss one, and a customer has to call again. We want to stop making customers chase work they have already requested.

Explain why you care

The work keeps slipping while people keep doing it by hand. Explain what waiting means for customers, the business, and trust in the team. Ask which part of the delay comes from scope, decisions, handoffs, or building the software.

Leave room for their answer

Ask what they would add or change. Try not to get too attached to your own idea. A smaller change, a different process, or a tool you haven't considered might help more.

Think beyond this one task

Make learning part of the reason to do this. What skills would people like to build for this job, their next role, or a side project? Ask them. Give that learning time and support, rather than treating it as something to squeeze in afterward.

Try: 'I can see why waiting matters. Could you share what you've heard from users? Then let's choose something worth trying and make room for the team to learn it properly.'

Reading and reflection: see the companion at the end of this Guide >

What would make this worth building?

Ask your PM what they hope will be better. Say what you would find useful and what you would need to trust the result.

Start with the problem

Which part of the work is causing trouble? Could a simpler change solve it? You can explore the problem without agreeing to a particular tool.

Make the work visible

Explain where time goes today: decisions, building, review, fixes, or support. Ask which tools or process changes might help. A useful demo can teach you something without answering every production question.

Say what would convince you

Name the evidence you would need and what would change your mind. Separate a risk you can investigate from a boundary you cannot approve changing.

Offer a way forward

You might recommend a smaller experiment, preparation, another approach, or stopping this use. Explain why and invite the people affected to help shape what happens next.

Stay open to a better approach. Make learning part of the purpose and fund it properly. The question is what helps users and the team, not whose first idea wins.

USE THESE QUESTIONS IN YOUR LAB >

Reading and reflection: see the companion at the end of this Guide >

Are we willing to try a different way?

You may be talking about more than one AI tool. Reports of big gains elsewhere can make the current pace feel hard to accept. They give you a reason to investigate, not a promise about what your team will achieve.

Name the real ask

“Are you willing to explore different tools or ways of working, to see whether we can deliver useful things sooner?” You are asking for a willingness to learn. The team should help choose what to try.

Ask what the first objection leaves out

“What worries you most about where this is going?” It might be review work, trust, job security, or losing a part of the craft they enjoy. Don’t decide that for them, and don’t dismiss a technical concern as an emotional one.

Talk about what good work means

A tool you expect to replace soon still needs to be safe and useful for its purpose. Ask what deserves lasting investment and what can stay simple. Engineering judgment matters in both; “good enough” needs a shared meaning.

Make room for both people to move

The PM may need to change the idea, scope, or deadline. The lead may see a process worth changing or a tool worth trying. Ask what would change each person’s mind, including what would make you stop.

If the answer is no, understand the no

Is it this tool, this task, the timing, or experimenting at all? What alternative would help the team improve? Name the tradeoff and who can decide if you still disagree. Don’t turn a conversation into a loyalty test.

Try: ‘I care about the quality of what we build. I’m willing to explore a different approach. Could we agree what we want to learn and what support the team gets?’

Reading and reflection: see the companion at the end of this Guide >

Before you talk with your product manager.

Take five or ten minutes before the conversation. You don't need a perfect argument. You need to know what matters to you and what you still need to ask.

What happened?

Describe what was said or done, using words the other person would recognize.

What am I assuming?

Notice the motive you have assigned to them. Depending on your history together and how you react under pressure, this can be hard. Write what you know, then what you need to ask.

Why does this matter to me?

What parts of engineering give you pride? What do you worry about losing, and what would you like to learn? Those questions can sit alongside concrete concerns about the software. You don't have to speak for everyone on the team.

What have I actually seen?

Separate something you observed from something you expect. Both can matter, but they are different kinds of information.

What would change my mind?

Name something that would make you reconsider. Be honest if you are attached to the tool or to the familiar way of working. Neither gets a free pass.

What do I still need to ask?

Write the question you haven't answered by yourself. That's a good place to start the conversation.

Private notes stay only in this open tab. They aren't saved or included in shared documents, and they disappear when you refresh or leave. You can also reflect without typing.

Reading and reflection: see the companion at the end of this Guide >

Ask what is behind your PM's request.

Your PM may be hearing about costs, commitments, and customer problems you haven't seen. Ask for that context. Explain the dependencies, support work, and choices that affect what the team can deliver.

What happens while we wait?

In our fictional example, support staff keep copying repair requests twice. Find out how often, what gets missed, and who it affects. Ask what makes a date meaningful. Delays matter, but urgency doesn't tell you which tool will help.

What is the opportunity worth?

Ask who owns the benefit estimate and what it assumes about adoption. Estimated annual business value divided by total team hours can help compare options. It is not hourly revenue or a measure of an individual developer. The value-and-effort chapter shows an example.

What could each person learn?

Make development a central reason for the work. Ask what skills people want for this role, a future one, or a side project. Plan time to learn and share what works. Don't assume everyone wants the same future or promise a promotion.

What would make it possible?

Ask for the time, tools, token budget, training, and experienced help the team needs. Name what other work will wait. A request to improve how we work needs support from the people setting priorities.

Try: 'Help me understand what waiting means for users and the business. I'll walk you through what is slowing us down. Then let's choose something we can learn from.'

Reading and reflection: see the companion at the end of this Guide >

Match the checks to what you're trying.

Once you have a specific experiment, decide what it needs. A prototype to learn about a workflow and a change to live customer software have different responsibilities. Be clear which one you are discussing.

Name the question and the finish line

Are you learning whether users understand a workflow, whether a tool helps build it, or how to improve review? Pick a manageable question. Say what the result must show and what it will not establish.

Choose the environment and information

Agree the tool, data, and access it needs. For an isolated prototype, made-up records may be enough. If real systems or records are involved, bring in the people responsible before connecting them.

Let the lead define the technical checks

For work involving protected records, define allowed and blocked access and test it at the server or API. Signing in and hiding buttons are not enough. For a prototype, state clearly which production checks remain outside this experiment.

Keep the next decision visible

Agree who can accept the experiment, where its use ends, and what would require another decision. Don't let a useful demo quietly become a live service. Don't require a finished service just to learn from an isolated demo.

OWASP: authorization checks

These are setup questions for the task you choose. They don't need to be the opening conversation about whether you're willing to explore a better approach.

Reading and reflection: see the companion at the end of this Guide >

Make it possible to learn and do good work.

Help the team choose what to practise and what support it needs. Name gaps without turning the experiment into a test of anyone's worth. You can care about the craft and explore new ways to do it.

Pair on a real, small piece of work

Arrange time with someone who knows the tools and the work. Walk through a change together: what it does, how you check it, and what you would do if it failed. The purpose is to build confidence and find where help is needed.

Help people get through reviews

Try useful automation alongside human judgment: tests, static checks, or an AI review tool. For example, Copilot can suggest issues and fixes on pull requests. Check whether the suggestions help and what they miss; budget the tool and human review time.

Make the route to acceptance clear

Agree who reviews, which checks apply, and how to ask for help before work begins. A pull request is a proposed code change. Keep it small enough to understand. Give feedback people can act on, and include the person doing the work in the conversation.

Look at the process as well as the tool

If work stalls, ask where: unclear scope, waiting for a decision, learning a new system, review, or rework? The familiar process deserves examination too. Use what you find to improve it, not to build a case against a colleague.

GitHub: Copilot code review

GitHub: responsible use of Copilot agents

DORA: working in small batches

Leave with a practical agreement: who will help, time and budget set aside, a task to learn on, and a date to compare what happened with what you hoped.

Reading and reflection: see the companion at the end of this Guide >

Give the conversation somewhere to go.

Use these questions with your colleague. Skip what you already understand. You are trying to decide what to do next; you do not need to answer every question.

1 / What matters?

Who are we trying to help, and how? What have you heard that I might not know?

2 / What needs protecting?

What worries you most about this change? Is it this task, the workload, what happens to the craft, or something else? Ask rather than assume. Take the technical concerns seriously too.

3 / What do we know?

What have we seen here, and what have we only heard about elsewhere? What would change either person's mind? What does each teammate want to learn?

4 / What is useful now?

Do we need another conversation, help from someone else, preparation, or a small experiment? Could we decide from what we already know? Who will do what next, and when will we talk again?

Before you finish, check what you've agreed and what's still a proposal. If something is unanswered, say who will follow up and when. Silence doesn't mean yes.

USE THESE QUESTIONS IN YOUR LAB >

Reading and reflection: see the companion at the end of this Guide >

What would be worth finding out?

Work out the experiment with your PM. Help choose the task, tool, and review approach. Ask for the business context and agree time, training, and a usage budget. You can propose a different approach if it would teach you more.

Choose a question worth answering

Could a different tool or process help us get a small repair-request workflow in front of users sooner? Could we make review easier too? Choose one question, then a task that can answer it.

Compare fairly

Record how similar work gets done today, including waiting and rework. Try the new approach on a comparable slice. Note differences in scope, experience, and checks. A prototype and a production release aren't like-for-like.

Match the checks to the task

Agree the tool, data, environment, and limits with the people responsible. An isolated prototype with made-up records needs a different plan from a change to live customer systems.

Budget time, tools, and support

Include product, design, engineering, review, and user feedback. Agree a token or usage budget, training or pairing time, and what work will wait. Nobody should have to learn it in their evenings.

Decide what would make it worthwhile

Agree what users should be able to do and what the team should learn. If saving time is the question, set a meaningful comparison. Choose stop conditions and a time to review what happened.

Give the team a clear path

Name who helps, who reviews, and who can accept the work. Let the people doing it shape the plan. Make the first step small enough to learn from and clear enough to start.

Choose how much you are willing to invest before starting. Review the result together before spending more. An experiment that never started has no result.

Reading and reflection: see the companion at the end of this Guide >

What will we try, and how much time will we spend?

Fictional proposal: explore one repair-request workflow with made-up records. Agree the people, time, tools, and review approach before starting. This is a learning prototype, not a live release.

What we expect

Could a different tool and review approach get useful feedback sooner? Ask Support to complete the task. Compare with a similar recent prototype, and note differences in scope, experience, and checks.

Time and money we could invest

Twelve total team hours, including up to three for review, over one week. At an assumed $150/hour plus $100 for tools and tokens, that is $1,900. Agree what other work waits.

What is allowed

Use made-up records in an approved, isolated environment. Choose the tool and review help with the lead. No live records, connections, or release. Name who checks the work and who can accept it.

What would count as useful

Users can complete the task, the agreed checks pass, and the team can explain what it learned. Stay within the time and usage caps. Decide together whether the learning justifies another step.

The next chapter connects estimated business value with total team effort. Review this plan with the people doing and approving the work; agreement covers this experiment only.

USE THESE QUESTIONS IN YOUR LAB >

Reading and reflection: see the companion at the end of this Guide >

What value could each team hour create?

Help your PM estimate the whole team's effort, including review and fixes. Ask how the expected business value was estimated. These invented numbers compare options; they don't rank developers or prove an AI productivity gain.

Illustrative estimate	Current approach	Possible approach
Annual business value	$180,000	$180,000
Total team hours	600 hr	400 hr
Annual value / team hour	$300 / hr	$450 / hr

Make the assumptions visible

The hours include engineering, design, product, and review. Both options assume the same useful outcome. The 400 hours is a hypothesis to investigate. Check the value with its owner, including adoption, scope, and shared foundations.

Price the experiment separately

Twelve team hours at an assumed blended $150/hour, plus $100 for tools and tokens, gives a $1,900 budget. At that rate, freeing about 12.7 team hours would equal that investment in capacity. It is not cash payback.

Then check what actually happened

Did users use it? Did it help? Count checking, fixes, and later support as well as drafting. Record elapsed delivery time separately. The Lab's minutes table can help compare one task; it cannot establish the product's annual value.

Keep estimated value, measured benefits, time freed, and cash savings separate. Include ongoing costs when deciding whether to continue. Never add overlapping benefits together.

Reading and reflection: see the companion at the end of this Guide >

What did we learn, and what should we do next?

Let's compare what happened with what you hoped to find. It's useful to learn that this task needs a different approach, or that this use of AI isn't worth continuing.

CONTINUE

The result supports using AI for this task under the agreed conditions. Say where that permission ends and when you'll check again.

REVISE

Something looks useful, but the task, instructions, or review needs to change. Describe the change before asking people to try again.

STOP

The result doesn't justify this use, or an agreed limit was broken. Explain why you're stopping and keep what you learned.

PAUSE

You're missing permission, information, time, or a person who needs to be involved. Name what is missing, who will resolve it, and when you'll return. You can choose this before an experiment.

Write what you decided, why, who will do what next, and when. If you haven't agreed, keep the record marked as a proposal and say what still needs deciding.

Reading and reflection: see the companion at the end of this Guide >

Make the result useful to other people.

Explain your findings and recommendation in your own voice. Make clear what your team can take responsibility for, what remains uncertain, and whose decision is needed next.

For leadership or your sponsor

Explain the problem, what you propose or have agreed, and what you know so far. Ask for a specific decision or help, name who can provide it, and say when you need an answer.

For your team

Tell the team why this matters, what work changes, and what stays the same. Name who will do what, what you still need to ask them, and when you will talk again.

Quality evidence

For each check, say what should happen, what actually happened, who checked it, and where the result can be seen. Mark it passed, failed, or not checked. Saving time does not show that the work is correct.

Come back and see what helped

Come back after the next action. Did it help? Do people need a different rule, more review time, training, or clearer permission? Decide who will do what next and when.

Explain what happened, what it tells you, and what you still do not know. Say who can approve the next step.

USE THESE QUESTIONS IN YOUR LAB >

Reading and reflection: see the companion at the end of this Guide >

From a stalled debate to a useful proposal.

Fictional case: your PM wants to explore AI because a replacement repair workflow keeps slipping. You see review and support work that a quick demo won't settle. You both want to find a useful way forward.

What the PM brings

"Support keeps asking when this will be ready. Here's what waiting means for them. I'm interested in the gains other teams report, but I don't know what we could achieve here. Are you willing to explore that with me?"

What the lead brings

"I'm open to trying. I don't want us to call a demo finished and leave the team to pick up the pieces. Let's choose a small workflow, make room for review, and get some help with the tools."

What they propose

A one-week prototype using made-up repair requests. Reserve twelve total team hours, including user feedback and review, plus $100 for tools and tokens. The PM asks what other work can wait; the lead arranges an experienced partner.

What they want to learn

Can support staff complete the task without a second entry? What slows the team down, and can tooling or a different review process help? Ask each person what skill they want to practise. A prototype is not a release approval.

What they tell leadership

"Could we make room for this experiment? The proposal is twelve team hours plus $100 for tools. We'll bring back user feedback, what the team learned, and a recommendation next Tuesday. We haven't agreed the time or started yet."

PAUSE until the people and budget are agreed. The next move is specific and reviewable. They haven't measured an AI productivity gain.

Reading and reflection: see the companion at the end of this Guide >

Bring one real conversation.

Ready to try this with your own situation? The Lab helps you work through the questions and prepare something to talk over with your colleague. Start where it helps; you don't need to fill every box.

1. Before you talk

Write private notes, or think through the questions without typing. What matters to you, what are you assuming, and what do you still need to ask?

2. Conversation

Share what you want to improve and ask about the concerns. Choose what to do next. Preview Next Step Together, check it, then copy, download, or print it. Mark a proposal as a proposal.

3. Experiment, if useful

Agree the task, time, checks, permissions, people, and review date. Read through the Experiment Plan before starting. You can skip this step if your next move doesn't involve an experiment.

4. Results

Return after the experiment. Enter all the time spent, the quality checks, and what happened. Preview the Results Record. Leave unknown values blank.

5. Next step

Write the decision and the reason. Name who will do what next and when you'll check again. Preview the Decision Brief and review it before sharing.

Shared drafts are saved in this browser on this device. Download work you need to keep before changing devices or clearing browser data. Private notes are never saved. Load Examples offers fictional answers; if the step contains work, it asks before replacing it.

Reading and reflection: see the companion at the end of this Guide >

What will you take into the next conversation?

After the immediate decision, take a moment to notice what changed. You may understand the problem, your colleague, or your own reaction differently now.

Identity / your judgment

What did you change your mind about? What do you understand now that you did not understand before?

Relationship / the conversation

What did you learn about what matters to your colleague? What helped you both think more clearly?

Vision / the worthwhile future

What would you like to make possible together? Why does it matter, and what would help you get there?

Culture / the conditions

What could change in how the team works so other people have the help, permission, or time they need too?

Choose one source below if it helps with a question you still have. Try an idea in your work, then return to see what you learned.

Agentic Engineering Patterns >

Shape Up: Write the Pitch >

OWASP Authorization Cheat Sheet >

OPEN THE TOOLS & STUDY LIBRARY > | SOURCES REVIEWED 12 OCT 2026

Reading and reflection: see the companion at the end of this Guide >

The thinking behind the tool.

Holistic Shaping asks what work is doing to people, as well as what people are doing at work. These short notes connect the Guide to Nic's teaching and a few practical sources. Read what helps with the situation in front of you. Links open the original resources.

Back in the Guide

CHAPTER 01

Two people are being shaped by this conversation.

You may leave an AI discussion feeling heard, dismissed, more curious, or more certain that the other person is the problem. That matters beyond the next experiment. Ask what this repeated argument is teaching each of you about your place on the team.

Worth asking: What would help both of you take part without having to defend your worth?

The Hollow Crown: Why Our Leadership Narratives Have Failed
Nic Payne · Article

CHAPTER 02

Let their answer change the future.

"Wouldn't it be cool if…"

Nic Payne · A future you can imagine together

In Nic's orbital model, vision grows through relationship. Bring your idea for what could be better and invite your colleague to change it. An invitation to help shape the future needs room for an answer you didn't expect.

Worth asking: Which part of your proposal are you willing to rethink together?

You Have a Culture Problem
Nic Payne · Article

The Lie We've Normalized: "Leadership Is Influencing Behavior"
Nic Payne · Article

A future you can imagine together
Nic Payne · Substack Note

CHAPTER 04

Meet the person behind the position.

Before opening the conversation, notice what you are listening for. Are you trying to catch a weak argument, or understand the person who has to live with this decision? Nic's bowl metaphor starts with the attention we bring. You are part of the relationship being shaped, too.

Worth asking: What might you hear if you stop preparing your reply for a moment?

Leadership as bowl, water, and current
Nic Payne · Video · start at 4:19

Every leader needs contact lenses
Nic Payne · Substack Note

The Mirror Move
Nic Payne / Rebel Leaders · Codex practice

CHAPTER 07

What will this way of working do to the team?

Holistic Shaping asks what our way of working does to people. Here, look beyond the code: room to learn the craft, confidence to question a result, a real say in the plan, work people can stand behind, and time to recover. Faster output can hide costs in any of those places.

Worth asking: What would you change so people become more capable rather than quietly carrying more?

When Mattering Isn't Enough
Nic Payne · Article

CHAPTER 08

Make the investment explicit.

Ryan Singer's Shape Up separates the problem worth solving from the amount of time you are willing to invest. That distinction can help you describe this experiment. It does not turn an untested idea into an approved production change.

Worth asking: What are you willing to spend to answer this question, including review?

Shape Up — Chapter 6: Write the Pitch
Ryan Singer · Book chapter

CHAPTER 09

Keep the evidence in its setting.

METR's early-2025 study and its 2026 update show why an AI productivity headline needs context. Teams, tasks, tools, and the way work is selected matter. Use the studies to sharpen your questions; use your own records to decide what happened here.

Worth asking: What did your comparison count, and whose work could it have missed?

We are Changing our Developer Productivity Experiment Design
Joel Becker, Nate Rush, Tom Cunningham, David Rein, Khalid Mahamud / METR · Research update

Early-2025 developer productivity study
METR · Research paper

CHAPTER 10

Keep what you learned together.

Record the reason for continuing, changing, pausing, or stopping. Also ask what changed between you: can people raise a concern sooner, explain their judgment, or offer a better idea? Those experiences help shape the next conversation and, over time, what feels normal here.

Worth asking: What should the next person be able to understand without repeating this argument?

You Have a Culture Problem
Nic Payne · Article

Do Your Replacement a Favor
Nic Payne · Article

Explore more of Nic's Holistic Shaping work at Rebel Leaders.

The notes apply the teaching to these situations. They are not quotations unless marked. The health reflection is optional; you do not need to record or share personal information.